What Happens When "Hacking" Becomes Easy? Teaching Python in 2025

Calado, Filipa
fcalado@pratt.edu
Pratt Institute, School of Information

Smyth, Patrick
patricksmyth01@gmail.com
Chainguard

Zweibel, Stephen
srzweibel@gmail.com
The Graduate Center, CUNY

Portela, Rafael Davis
rdportela@gmail.com
The Graduate Center, CUNY

In the "big tent" days of the Digital Humanities (DH), debates over "hack versus yack" and whether DH scholars should always be "builders" underscored the field's tension between technical expertise and interpretive scholarship. Since 2022, however, the emergence of AI coding tools is reshaping the stakes of this conversation. Writing Python scripts no longer requires years of practice; now, a well-crafted prompt can produce code in moments. While programming appears more and more attainable for non-technical users, especially those who lack the necessary resources for learning how to code, other considerations may be falling to the wayside. As many have pointed out, when a tool automates a task (e.g., data cleaning), users may not notice its assumptions or limitations, leading to overly simplistic interpretations of complex phenomena.

This shift poses urgent questions about how Python education and research projects should adapt to this new landscape. What happens when hacking becomes easy? Does the streamlining of technical processes diminish the intellectual labor of coding, or does it free scholars to focus on the interpretive and conceptual dimensions of their work? If traditional coding education involved mastering challenging skills and overcoming high barriers to entry, what new forms of rigor emerge when these barriers are lowered?

This panel explores the implications of this paradigm shift from the perspective of four seasoned Python instructors, whose roles span graduate student, professor, librarian, and industry professional. While some of the perspectives emphasize the importance of foundational coding skills for cultural and feminist critique, others advocate for flexibility in evolving with technological changes that lower barriers to creating and sharing work. The panel calls for a reconsideration of what it means to teach, learn, and critique code in an age of AI assistance, ensuring that DH scholarship remains both accessible and intellectually rigorous. Together, these perspectives highlight the evolving role of Python—and programming more broadly—in DH.

PAPERS

Dr. Filipa Calado is an Assistant Professor at the Pratt Institute School of Information. Her presentation explores how AI technology can be re-purposed not to automate or streamline tasks, but to engage directly with underlying biases that drive these tools. She argues that close attention to the mechanisms of coding and the assumptions that circulate within computational processes can illuminate how bias operates in social and discursive contexts more broadly.

Filipa deploys AI to interrogate its own biases in her research project, which uses Large Language Models (LLMs) to study discourses of transphobia in the US. For this project, she trains an LLM with examples of transphobic text, culled from current “anti-trans” legislative bills that are proliferating across the US, with the purpose of examining the bias and discrimination that result in its output. Each step of data gathering and model development opens the logics and assumptions behind machine learning processes to critical analysis which can lead to surprising realizations. For example, prediction algorithms, which turn semantic meaning in language into numerical probabilities, what Filipa calls a “regularization” or “approximation” of language, reveals unexpected commonalities between polarized political perspectives, surfacing shared investments across transphobic and gender-affirming positions. In this context, AI tools are deliberately deployed not for efficiency or productivity, but as a means of turning them back on themselves, offering new objects and rich opportunities for critical analysis.

Dr. Patrick Smyth is a staff engineer and training specialist at a top five venture-backed security startup. Patrick argues for a renewed focus on the fundamentals of coding as essential to cultural criticism. He asserts that a deep understanding of programming principles allows DH scholars to interrogate the assumptions embedded in computa-

tional tools, from the algorithms driving data analysis to the biases encoded in AI systems.

Patrick argues that large language models are fundamentally backward-looking— lowering the cost to produce work based on previous inputs moves us toward a conservative, stultifying, and ultimately mediocre technocultural environment. Further, the development of technologies such as LLMs have fractured the interfaces, such as public APIs, that enabled productive tinkering and remixing, as organizations such as the New York Times or Reddit are racing to close the gates on an outflow of information to AI companies. While code can be quickly written, that easily-written code rarely matters, as algorithmic and platform incumbents tighten the screws on communities of practice. Ultimately, AI saps energy from communities of builders, creating a new set of intermediaries and impediments between communities of practice and meaningful change. Patrick believes that, while AI is here to stay, programmers must reinforce concepts such as platform autonomy, interoperability, cultivated illegibility, and disintermediation.

Stephen Zweibel is Digital Scholarship Librarian at the CUNY Graduate Center. He supports digital project creation by GC researchers across disciplines , helps preserve those projects , and supports faculty and students with their data-based research and data management needs .

Co-programming with AI assistants reframes how humanities students learn to code: the emphasis moves from rote syntax to directing, critiquing, and refining machine-generated code. By shifting from local Python setups to browser-runnable JavaScript apps—and deploying via GitHub Pages or Observable—students bypass installation hurdles and immediately share live projects. First, learners build foundational JS fluency (control flow, DOM manipulation, data structures); then they integrate LLM tools (e.g., CLine, Roo Code) to scaffold web applications. Each generated snippet becomes an object of critical analysis—students interrogate design choices, expose biases, and iterate accordingly. This two-stage model preserves scholarly rigor while democratizing web-based DH practice: LLM-enhanced co-programming extends, rather than replaces, humanistic inquiry.

Rafael Davis Portela is a Ph.D. Candidate in History at the CUNY Graduate Center, researching the History of Computing and of the Computer Industry.

Rafael points out that what and how we teach is always changing, and this is not something that AI is forcing us to deal with for the first time, although LLMs might force us to change faster. While we are in the middle of a transition, and don't know exactly how DH and computing at large will look like in the future, we know we have to adapt. It is a great time to experiment and to force ourselves to try different things. At the same time, we also need to be cautious and recognize that technological changes are often overstated in what they will do in the short term. We should avoid rushed conclusions and the urge to throw away pedagogical practices we have developed over the past decade. Situating his comments in a larger context of the History of Computing, Rafael concludes that Python is, after all, still a dominant language and understanding code continues to be incredibly important in this post-LLM world.